



Complete Build Plan

From Repository to Production

A zero-overhead CLI utility that prepares software repositories for agentic AI tools.
Covering architecture, implementation, testing, ecosystem, and production operations.

Detector Engine

Validator Engine

Translator Engine

Template Registry

CI/CD Integration

Enterprise Features

14

Sections

4

Engines

12wk

Roadmap

5

Channels

95%+

Coverage

Table of Contents

- 01** Executive Summary & Value Proposition
- 02** Problem Analysis & Market Context
- 03** Architecture Deep Dive
 - Four Engines: Detector · Validator · Templater · Translator
- 04** Repository Detection Heuristics
 - Full language, framework, and tooling signal matrix
- 05** Template System & Pack Registry
 - Handlebars engine, idempotency, custom packs
- 06** Validation Pipeline
 - Structural · Semantic · Logical · Drift engines
- 07** Backend Translation Layer
 - IR schema, adapter pattern, conversion fidelity
- 08** CLI Specification
 - Full command surface, flags, exit codes, UX
- 09** Developer Experience & Extensibility
 - Plugin API, hook authoring, IDE integration
- 10** Implementation Roadmap
 - 12-week phased plan with milestones and deliverables
- 11** Technology Stack & Data Flow
 - Stack rationale, data flow, file I/O strategy
- 12** Production Operations
 - CI/CD, distribution, performance targets, security model
- 13** Testing Strategy
 - Test pyramid, fixture repos, fuzz testing, performance
- 14** Gaps Addressed & Future Directions
 - Completeness audit, open questions, v2 roadmap

What is blueprint?

blueprint (bp) is a zero-runtime-overhead CLI utility that prepares software repositories for agentic AI tools — Claude Code, Cursor, OpenDev, Goose, and others — by scaffolding governance structures and verifying their integrity. It operates entirely at **development-time and CI-time**, never at runtime. Once bp finishes, it may be uninstalled and the generated configuration continues to function natively inside the target agentic tool. bp is a *scaffolding generator and integrity verifier*, not a middleware layer.

The Zero-Overhead Contract

Every design decision in bp follows the zero-overhead contract: generated files are plain static configuration files that agentic tools already consume. bp adds no runtime tokens, no latency, and no context window pollution to the agentic loop. Running bp init generates files; after that, the agentic tool reads those files directly — bp is not involved.

Core Value Proposition

Pain Point	bp Solution
Agentic tools silently ignore malformed config files	Structural + semantic validation with actionable, line-precise error reporting
Teams manually duplicate governance across 50+ repositories	Idempotent, templated scaffolding from a shared, versioned registry
Rules contradict each other across directories and scopes	Scope-intersection analysis detects logical conflicts before execution
Repository drift breaks agentic conventions silently	Automated fingerprinting and drift detection via CI or local dev
Vendor lock-in to a single agentic tool format	Backend-agnostic IR with format translation between all supported tools
No visibility into whether the agentic tool is reading config correctly	bp doctor command provides full diagnostic trace and schema conformance check
Org-wide policy enforcement is manual and error-prone	Blueprint inheritance: repo blueprint extends org base template

Guiding Architectural Principles

- **Scaffolding-Only:** bp runs once at setup or in CI. It never intercepts the agentic loop at runtime.
- **Idempotency:** Running bp init twice on the same repo produces the same result; user edits in preserve blocks are never overwritten.
- **Backend-Native Output:** Generated files conform exactly to the target tool's documented schema — no abstraction layers survive into runtime.
- **Semantic Preservation:** A blueprint's meaning (rules, constraints, skills) is tool-agnostic; only the file format changes across backends.

- **Fail-Loud Validation:** bp errors are always actionable: every diagnostic includes the file, line, the problem, and at least one resolution path.
- **Progressive Disclosure:** The simplest usage (bp init) works with zero configuration; advanced features unlock incrementally via flags and .bp.json.

The Agentic Configuration Gap

Current agentic coding tools provide rich extension surfaces, but none provide validation, conflict detection, auto-detection of repository structure, or portability between tool formats. This creates a governance vacuum where teams write rules that are silently ignored, malformed, or contradictory.

Tool	Extension Surface
Claude Code	.claude/rules/*.md, .claude/skills/*.md, .claude/agents/*.md, hooks, CLAUDE.md
Cursor	.cursor/rules/*.md, .cursor/agents/, context files
OpenDev	.opendev/, skill registry, MCP tools
Goose	Profile YAML, extensions, context files

None of these tools provide:

- Validation of the config files they consume
- Conflict detection between overlapping rules
- Auto-detection of repository structure to generate sensible defaults
- Portability between tool formats
- Drift detection when the repository evolves
- CI-enforced blueprint integrity checks

The Drift Problem

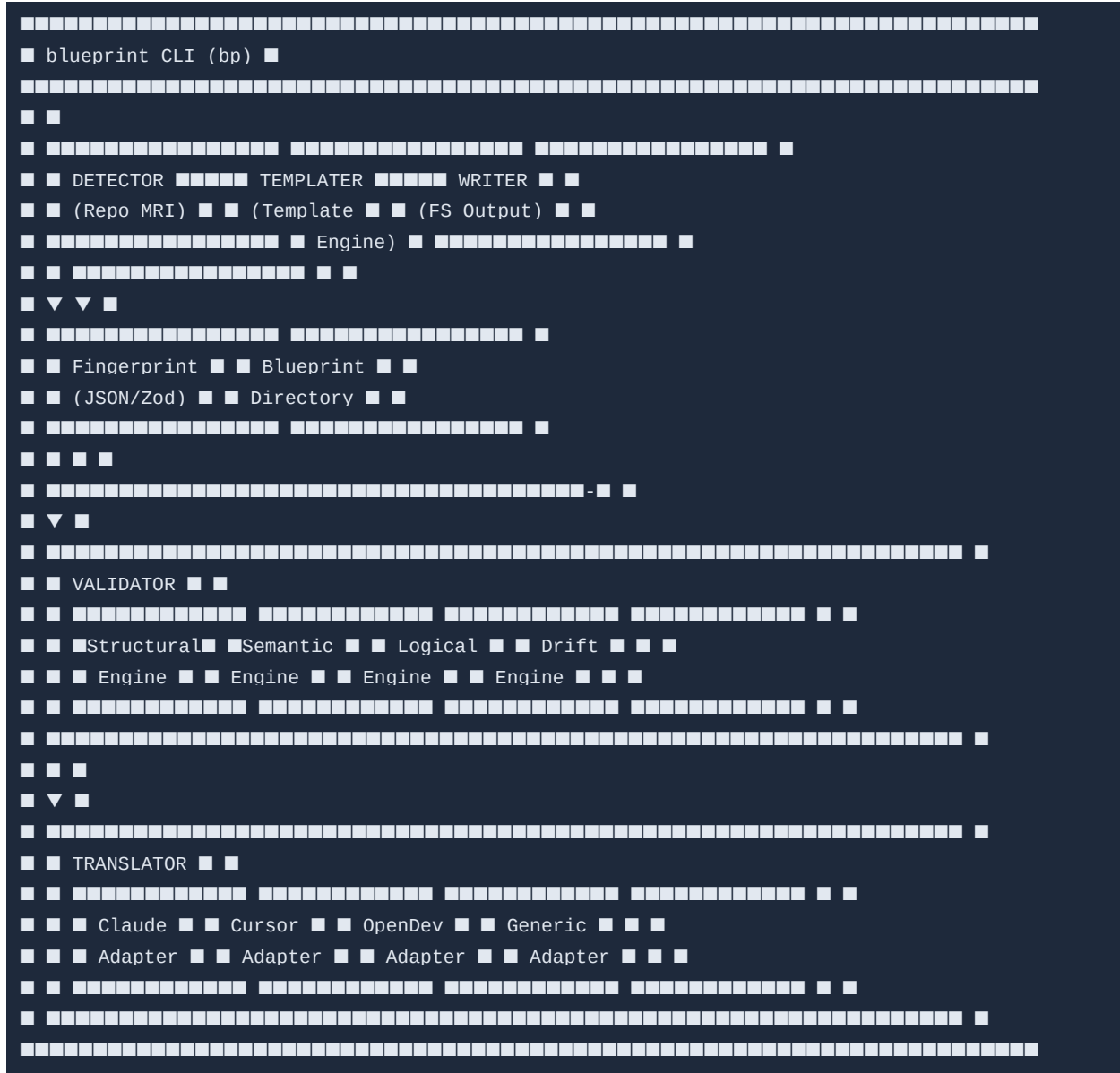
Repositories evolve continuously. Entry points move, test runners migrate (Jest → Vitest), new directories appear, package managers switch. Agentic configuration becomes stale silently. Without a verification mechanism, the AI operates on outdated assumptions:

- Incorrect test commands referenced in skills
- Missing coverage for newly created source directories
- Orphaned skills referencing deleted tools or scripts
- Style rules that contradict the actual linter configuration
- Agent tool-allowlists that block newly required capabilities

The Scale Problem

Organizations with 50+ repositories face an infeasible manual governance burden. Each repo needs spatial anchors, security constraints, style conventions, and capability skills. bp treats blueprints as infrastructure-as-code: templated, versioned, and CI-enforced.

System Architecture Overview



Engine Responsibilities

Engine	Responsibility
Detector	Static-only repository analysis — builds a deterministic fingerprint in milliseconds without executing build commands
Templater	Maps fingerprint to template packs; renders backend-native configuration files using Handlebars
Validator	Four-layer pipeline: structural, semantic, logical, and drift checks with actionable error reporting

Engine	Responsibility
Translator	Converts blueprints between backends via an Intermediate Representation (IR) without semantic loss

The Five Blueprint Layers

Layer	Name	File Pattern	Purpose
Layer 1	Spatial Anchor	CLAUDE.md / context.md	Where is the agent in the project lifecycle? What surface is active?
Layer 2	Personas / Agents	.claude/agents/*.md	Who is the agent acting as? Planner, Implementer, Reviewer personas.
Layer 3	Rules	.claude/rules/*.md	What must / must not happen? Hard constraints and soft style conventions.
Layer 4	Skills	.claude/skills/*.md	Reusable capability units: how to perform a specific task.
Layer 5	Hooks	.claude/hooks/*	Lifecycle callbacks at tool-use boundaries (pre/post tool use).

Detection Philosophy

The Detector uses only static file analysis — no shell commands, no network calls, no build-tool invocations. This ensures sub-second detection in CI environments without any build dependencies. All signals are weighted and combined into a confidence score per dimension.

Language Detection Signals

Language	Signal Files	Confidence
TypeScript / JavaScript	package.json present; .ts or .tsx files in src/	High
Python	requirements.txt, setup.py, pyproject.toml, Pipfile, poetry.lock	High
Go	go.mod, go.sum present	High
Rust	Cargo.toml, Cargo.lock present	High
Java / Kotlin	pom.xml, build.gradle, .java or .kt files	High
Ruby	Gemfile, Gemfile.lock, .rb files	High
Dart / Flutter	pubspec.yaml, lib/main.dart	High
C / C++	CMakeLists.txt, Makefile, .cpp / .c files	Medium
C#	.csproj, .sln files	High
Swift	Package.swift, .xcodeproj	High

Framework Detection Signals

Framework	Detection Signal	Confidence
Next.js	next in dependencies, next.config.*	High
React	react in dependencies, jsx/tsx files	High
Express	express in dependencies, app.js / server.js	High
FastAPI	fastapi in requirements, main.py with FastAPI()	High
Django	django in requirements, settings.py, manage.py	High
Spring Boot	spring-boot-starter in pom.xml or build.gradle	High
Axum / Actix	axum or actix-web in Cargo.toml	High
Rails	rails in Gemfile, config/application.rb	High
NestJS	@nestjs/core in dependencies, src/main.ts	High
Vue / Nuxt	vue or nuxt in dependencies, .vue files	High

Framework	Detection Signal	Confidence
Svelte / SvelteKit	svelte or @sveltejs/kit in dependencies	High
Flutter	flutter in pubspec.yaml dependencies	High

Tooling Detection Signals

Category	Signal Logic
Package Manager	npm: package-lock.json yarn: yarn.lock pnpm: pnpm-lock.yaml bun: bun.lockb poetry: poetry.lock pip: requirements.txt
Test Runner	jest.config.* , vitest.config.* → JS; pytest.ini, conftest.py → Python; go test (go.mod) → Go; cargo test (Cargo.toml) → Rust
Build Tool	vite.config.* → Vite; webpack.config.* → Webpack; Makefile → Make; Dockerfile → Docker; bazel → Bazel; nx.json → Nx
Linter/Formatter	.eslintrc*, biome.json → JS; .prettierrc* → Prettier; pyproject.toml [tool.ruff] → Ruff; rustfmt.toml → rustfmt
Monorepo	packages/ dir + pnpm-workspace.yaml lerna.json nx.json turbo.json rush.json
CI System	.github/workflows/*.yml → GitHub Actions; .gitlab-ci.yml → GitLab; .circleci/ → CircleCI; Jenkinsfile → Jenkins
Git Workflow	CODEOWNERS → PR review; .github/PULL_REQUEST_TEMPLATE.md → PR template; release/* branches → gitflow

Fingerprint Schema (Zod-Validated)

```
const FingerprintSchema = z.object({
  version: z.literal("1.0"),
  detected_at: z.string().datetime(),
  project: z.object({
    name: z.string(),
    root: z.string().absolute(),
    type: z.enum(["monorepo", "polyrepo", "library", "application", "service"]),
    git_workflow: z.enum(["github-flow", "trunk-based", "gitflow", "unknown"]),
  }),
  languages: z.array(z.object({
    name: z.enum(["typescript", "javascript", "python", "go", "rust", "java", "ruby", "dart", "cpp", "csharp", "swift"]),
    confidence: z.number().min(0).max(1),
    primary: z.boolean(),
  })),
  frameworks: z.array(z.object({ name: z.string(), confidence: z.number() })),
  entry_points: z.array(z.object({
    path: z.string(), type: z.enum(["cli", "server", "library", "ui"]),
  })),
  tooling: z.object({
    package_manager: z.string().optional(),
    test_runner: z.string().optional(), test_command: z.string().optional(),
    build_tool: z.string().optional(), linter: z.string().optional(),
    formatter: z.string().optional(), ci_system: z.string().optional(),
  }),
  directory_topology: z.object({
```

```
src_dirs: z.array(z.string()), test_dirs: z.array(z.string()),
config_dirs: z.array(z.string()), package_dirs: z.array(z.string()),
}),
security_signals: z.object({ // NEW: security context for rules
has_auth: z.boolean(),
has_external_apis: z.boolean(),
has_secrets_manager: z.boolean(),
has_docker: z.boolean(),
}),
raw_signals: z.record(z.any()),
});
```

Template Engine Design

bp uses Handlebars as its template engine — logic-less, well-known, fast, and with zero runtime dependencies after compilation. Templates are markdown files with embedded Handlebars expressions. No arbitrary JavaScript may execute inside a template; custom helpers are allowlisted and audited.

Idempotency via Block Markers

```
<!-- bp-generated:begin position -->
# Position: {{project_name}}
- Language: {{primary_language}} ({{primary_framework}})
- Entry: {{entry_point_path}}
- Test command: {{test_command}}
<!-- bp-generated:end position -->

<!-- bp:preserve -->
# Custom team notes (NEVER overwritten by bp)
Our team uses specific naming conventions for services...
<!-- bp:end-preserve -->
```

Generated blocks are identified by begin/end markers and replaced on subsequent bp init runs. Preserve blocks are never touched. This allows teams to add custom content without losing it on updates.

Template Pack Directory Structure

```
templates/
├── _base/ # Shared partials (tool-agnostic)
│   └── partials/
│       ├── security-rules.md.hbs # Common security rules
│       ├── style-rules.md.hbs # Style conventions
│       ├── test-patterns.md.hbs # Testing patterns
│       └── helpers.js # Allowlisted Handlebars helpers
├── claude/
│   ├── manifest.json # Backend metadata & schema version
│   ├── CLAUDE.md.hbs # Spatial anchor template
│   └── agents/
│       ├── planner.md.hbs
│       ├── implementer.md.hbs
│       └── reviewer.md.hbs
├── rules/
│   ├── 01-position.md.hbs # Phase definitions
│   ├── 02-security.md.hbs # Hard security constraints
│   ├── 03-style.md.hbs # Soft style conventions
│   └── 04-meta.md.hbs # Rule precedence
├── skills/
│   ├── add-test.md.hbs
│   └── refactor-async.md.hbs
├── cursor/ manifest.json + cursor-native syntax
├── opendev/ manifest.json + opendev-native syntax
└── generic/ backend-agnostic markdown
```

Template Selection Logic

```
fingerprint → language → framework → template_pack
[node, ts] + [nextjs] → templates/claude/nextjs/
[node, ts] + [express] → templates/claude/express/
[node, ts] + [nestjs] → templates/claude/nestjs/
[python] + [fastapi] → templates/claude/fastapi/
[python] + [django] → templates/claude/django/
[go] + [std] → templates/claude/go/
[rust] + [axum] → templates/claude/rust-axum/
[java] + [spring] → templates/claude/java-spring/
Fallback chain: framework-specific → language-base → generic
```

Template Registry

Template packs are distributed via npm-compatible registries. Each pack is a signed npm package prefixed `@bp-templates/`. The bp CLI verifies signatures before extraction using the registry's public key. Organizations may host a private registry for internal packs.

- `bp template list` — list all available packs (official + installed)
- `bp template add` — install a local custom pack
- `bp template install @bp-templates/fastapi` — install from registry
- `bp template publish` — publish to registry (requires auth token)
- Pack versioning follows semver; bp respects version ranges in `.bp.json`
- Packs include a `manifest.json` declaring supported backend versions and features

Validation Architecture

Validation runs in four sequential layers. Each layer builds on the previous; a hard failure in an earlier layer short-circuits later layers for that file (though other files continue). All errors are collected and emitted together with file path, line number, error type, severity, and at least one suggested resolution.

Layer 1: Structural Engine

Check	Description
File Existence	Checks that all files declared in manifest patterns are present
YAML Frontmatter	Parses YAML in --- blocks; reports parse errors with line number
Frontmatter Required Fields	Validates required fields per backend manifest (e.g. scope, severity for rules)
File Size Limits	Enforces per-type size limits from backend manifest (e.g. rules ≤ 10 KB)
Markdown Well-formedness	Detects unclosed code fences, broken links, malformed heading hierarchy
Encoding	Validates UTF-8 encoding; rejects BOM-prefixed files

Layer 2: Semantic Engine

Check	Description
Scope Pattern Resolution	Resolves glob patterns against the actual filesystem; warns if a scope matches zero files
Tool Reference Validation	Checks that tools_required in skills reference valid agentic tool names
Agent Tool-Allowlist	Verifies that tools in agent allowed_tools exist and are spelled correctly
Cross-Reference Integrity	Ensures skills referenced by rules actually exist in the skills directory
Backend Schema Conformance	Validates enum values, optional field types, and nested schema conformance
Empty Rule Bodies	Warns if a rule has a frontmatter but no body content (vacuous rule)

Layer 3: Logical Engine

The most critical validation layer. Detects conflicts and contradictions that would silently cause the agentic tool to receive contradictory instructions.

Check	Description
Scope Intersection	Identifies pairs of rules whose glob scopes overlap using fast-glob + set intersection
Severity Conflict	hard vs hard intersection = ERROR; hard vs soft intersection = WARNING

Backend Translation Layer

IR schema · Adapter pattern · Conversion fidelity guarantees

Translation Strategy

The Translator converts a blueprint from one agentic tool format to another. The strategy uses a two-phase parse-then-render pipeline via a tool-agnostic Intermediate Representation (IR). The IR preserves all semantic content; only the serialization format changes per backend.

```
Phase 1: Parse
source backend files → IR
IR is validated with Zod for semantic completeness

Phase 2: Render
IR → target backend templates → target files
Output validated against target backend manifest schema

Round-trip guarantee:
claude → cursor → claude must reproduce the original blueprint
(tested via round-trip snapshot tests; fidelity target: >98%)
```

Intermediate Representation Schema

```
interface BlueprintIR {
  version: "1.0";
  spatial_anchor: {
    project_name: string; surface: string;
    temporal_anchor: string; conventions: string[];
  };
  personas: Array<{
    name: string; role: string; reasoning_style: string;
    constraints: string[]; allowed_tools?: string[];
  }>;
  rules: Array<{
    id: string; scope: string; severity: "hard" | "soft";
    action: string; rationale?: string; tags?: string[];
  }>;
  skills: Array<{
    name: string; description: string; when_to_use: string;
    tools_required: string[]; procedure: string;
  }>;
  hooks: Array<{
    event: "pre_tool_use" | "post_tool_use";
    language: string; stub: string;
  }>;
  meta: {
    rule_precedence: string[];
    conflict_resolution: string;
    source_backend: string;
    target_backend: string;
  };
}
```

Supported Translation Paths

Translation Path	Availability	Fidelity
Claude Code → Cursor	Available Phase 3	High — both use markdown rules
Cursor → Claude Code	Available Phase 3	High
Claude Code → OpenDev	Available Phase 3	Medium — skill format differs significantly
Claude Code → Generic	Available Phase 3	Full — generic is a superset
Generic → Any	Available Phase 3	High — generic preserves all semantics
Claude Code → Goose	Planned Phase 4	Medium — Goose uses YAML profiles
Any → Any (chained)	Planned Phase 4	Via IR hub; no information loss

Command Surface

```

bp [command] [options]

init [tool] Scaffold blueprint for current repository
--tool <backend> claude | cursor | opendev | generic (default: auto-detect)
--template <name> Use specific template pack (default: auto-detect)
--force Overwrite existing blueprint (preserves user blocks)
--dry-run Show diff of what would be generated
--no-verify Skip post-init validation run

verify Validate blueprint integrity
--level <level> structural | semantic | logical | drift | all (default: all)
--json Machine-readable JSON output
--fix Auto-correct unambiguous issues
--watch Watch mode: re-validates on file change
--fail-on <level> Exit non-zero only at this severity (default: logical)

sync Detect and resolve repository drift
--auto-apply Apply all safe fixes without prompting
--report Generate drift report only; no changes

convert Translate blueprint between backends
--from <backend> Source backend
--to <backend> Target backend
--input <path> Source blueprint directory (default: ./)
--output <path> Output directory

template Manage template packs
list List available templates (official + installed)
add <path> Install custom local template pack
install <pkg> Install from registry (@bp-templates/fastapi)
publish <path> Publish to registry (requires auth token)
update Update all installed packs to latest

doctor Diagnostic mode for troubleshooting
--tool <backend> Diagnose why a tool might ignore config
--verbose Full diagnostic trace with timing

rule Rule management utilities
test <file> Dry-run a rule against mock file scenarios
lint <file> Check rule syntax and scope pattern validity
graph Visualize rule scope coverage as ASCII map

hook Hook management
generate Generate hook stubs for current backend
validate <file> Validate hook safety (no network, no secrets)

config Configuration management
get <key> Read a config value
set <key> <value> Set a config value
reset Reset to defaults
update Update bp itself to latest version

```

Exit Codes

Code	Meaning
0	Success — all checks passed
1	General / unexpected error
2	Validation failed: structural (malformed files)
3	Validation failed: semantic (bad references, missing fields)
4	Validation failed: logical (rule conflicts, circular deps)
5	Drift detected (repo has changed since last fingerprint)
6	Unsupported backend requested
7	Template pack not found
8	Permission denied on file system
9	Registry unreachable (network error)
10	Signature verification failed on template pack

UX Design Principles

- Progress spinners with timing for operations > 200ms
- Color-coded output: errors in red, warnings in amber, success in green
- Always show the resolution path alongside an error — never just the problem
- Dry-run (--dry-run) available on every write command — shows unified diff
- JSON output mode (--json) for all commands for CI and tool integration
- Interactive prompts use ink (React-for-CLI) with keyboard navigation
- No required environment variables — all config via flags or .bp.json
- Auto-update notification: shows latest version on every run if outdated

Plugin / Extension API

bp exposes a stable TypeScript API for building plugins. Plugins can add custom validation rules, new backend adapters, or custom Handlebars helpers. Plugins are installed via the template registry and declared in .bp.json.

```
// Example custom validation plugin
import { definePlugin, ValidationContext } from "@agentic/bp/plugin";
export default definePlugin({
  name: "my-org-rules",
  version: "1.0.0",
  validators: [{
    id: "require-rationale",
    level: "semantic",
    check: (ctx: ValidationContext) => {
      for (const rule of ctx.blueprint.rules) {
        if (!rule.frontmatter.rationale && rule.frontmatter.severity === "hard") {
          ctx.error(rule.file, rule.line,
            "Hard rules must include a rationale field",
            "Add: rationale: 'Why this constraint exists'");
        }
      }
    }
  }]
});
```

Hook Authoring Guide

bp generates hook stubs — skeleton files that the agentic tool will execute at lifecycle boundaries. bp never executes hooks itself. Stubs are intentionally minimal and include inline comments explaining the hook contract.

```
// .claude/hooks/pre_tool_use.js (generated by bp)
// Called by Claude Code BEFORE each tool invocation
// Input: { tool_name, tool_input, session_id }
// Output: { allow: boolean, reason?: string }

module.exports = async function preToolUse({ tool_name, tool_input }) {
  // TODO: Add your validation logic here
  // Example: block writes to protected paths
  if (tool_name === "write_file") {
    const path = tool_input.path;
    if (path.startsWith("secrets/") || path.startsWith(".env")) {
      return { allow: false, reason: "Writing to secrets/ is blocked by blueprint" };
    }
  }
  return { allow: true };
};
```

IDE Integration (Planned Phase 4)

- VS Code extension: real-time blueprint linting as you edit rules/*.md files

- Inline diagnostics: red underlines on malformed frontmatter, bad scope patterns
- Quick-fix suggestions: one-click resolution for unambiguous errors
- Blueprint explorer: tree view of all rules, skills, agents with conflict highlights
- JetBrains plugin: same capabilities via Platform Plugin SDK
- LSP server (blueprint-lsp): language server protocol for any LSP-capable editor

Custom Backend Adapter

Third parties can implement new backend adapters for agentic tools not natively supported. An adapter consists of a manifest.json, a template directory, and optionally an IR parser for translation support.

```
// adapter/manifest.json (minimum required)
{
  "backend": "myagent",
  "version": "1.0",
  "supported_features": { "rules": true, "skills": true, "agents": false, "hooks": false },
  "file_patterns": { "rules": ".myagent/rules/*.md", "skills": ".myagent/skills/*.md" },
  "max_file_sizes": { "rules": 8000, "skills": 12000 },
  "frontmatter_schema": { "rules": { "required": ["scope"], "optional": ["severity"] } }
}
```

Phase 1: Core Foundation

Weeks 1–3 — Working bp init and bp verify for Claude Code backend.

- Project bootstrap: TypeScript + Bun, Zod schemas, Commander.js CLI, Biome linting
- Detector Engine v1: Node.js, Python, Go, Rust; file existence heuristics; lockfile parsing; directory topology scanning
- Templater Engine v1: Handlebars integration; base + Claude Code template pack; block-level merging for idempotency
- Writer: file system output with .blueprintignore support and --dry-run diff output
- Validator Engine v1: structural validation — frontmatter parsing, file size checks, required fields
- CLI: bp init --tool claude, bp verify --level structural
- Test suite: unit tests for Detector heuristics and Validator structural checks
- CI: GitHub Actions matrix for Node 20/Bun, lint, type-check, test

Phase 2: Integrity Engine

Weeks 4–6 — Full semantic, logical, and drift validation. bp verify catches all classes of issues.

- Detector Engine v2: framework detection (React, Next.js, FastAPI, Django, etc.); security signals
- Semantic Validator: scope pattern resolution; tool reference validation; cross-reference integrity
- Logical Validator: rule scope intersection algorithm; conflict detection; circular dependency detection
- Drift Engine: fingerprint storage (.bp-fingerprint.json); entry point, test command, directory topology drift
- bp sync command with interactive fix prompts (ink UI)
- bp verify --fix for auto-correction of unambiguous issues
- bp verify --watch for development mode
- Integration tests with 10 fixture repositories covering all supported language/framework combinations

Phase 3: Ecosystem & Distribution

Weeks 7–9 — Multi-backend support, template registry, and all distribution channels live.

- Translator Engine: IR schema, Claude → Cursor adapter, Cursor → Claude adapter, generic adapter
- bp convert command; round-trip snapshot tests
- Template Registry: npm-compatible registry; pack signing and verification; bp template commands
- Cursor and OpenDev template packs
- Distribution: npm package @agentc/bp, Homebrew formula, GitHub Releases binaries, Docker image
- Documentation: README quickstart, template authoring guide, backend adapter guide, CI examples
- bp doctor diagnostic command

Phase 4: Production Hardening

Weeks 10–12 — Enterprise-ready reliability, performance, observability, and IDE integration.

- Performance: parallel file scanning; incremental validation (only changed files); 10,000+ file repo support
- CI/CD integrations: GitHub Action `agentic-blueprint/verify@v1`, GitLab CI template, Azure DevOps extension, pre-commit hook
- Enterprise features: private template registry (S3/Artifactory), org-wide base templates, blueprint inheritance, audit logging
- Security hardening: template sandboxing, hook stub safety validation, secret scanning in generated files
- Fuzz testing (1000+ random repository structures); backward compatibility guarantee for blueprint schemas
- VS Code extension (`blueprint-lsp`): real-time linting, inline diagnostics, quick-fix suggestions
- 95% test coverage enforced via Codecov; 99%+ false-positive-free validation on fixture suite

Technology Stack

Component	Technology	Rationale
Runtime	Bun / Node.js 20+	Bun for fast compilation to single binary; Node for broad CI compatibility
Language	TypeScript 5.x	Type safety, ecosystem maturity, excellent tooling
CLI Framework	Commander.js	Mature, well-documented, sub-command architecture
Interactive UI	ink (React-for-CLI)	Component model for prompts, progress bars, spinners
Template Engine	Handlebars	Lightweight, logic-less, auditable — no arbitrary code in templates
Schema Validation	Zod	TypeScript-native, composable, excellent error messages for users
Glob Matching	fast-glob + picomatch	Industry standard; fast; supports negation patterns
Markdown Parsing	gray-matter + remark	Frontmatter extraction + AST for validation
Testing	Vitest	Fast, native TypeScript, excellent watch mode
Linting/Formatting	Biome	Unified formatter + linter; minimal config; fast
Build	bun build --compile	Single binary output; zero runtime dependencies
Logging	pino	Structured JSON logging for CI/observability integration
Error Formatting	pretty-format	Human-readable error output with color and context

Project Directory Structure

```

bp/
├── src/
│   ├── cli/ # Commander.js command definitions
│   ├── detector/ # Repository MRI engine
│   │   ├── languages.ts
│   │   ├── frameworks.ts
│   │   ├── tooling.ts
│   │   └── fingerprint.ts
│   ├── templater/ # Handlebars template engine
│   │   ├── engine.ts
│   │   └── helpers.ts
│   ├── merger.ts # Idempotent block merging
│   ├── validator/ # Four-layer validation pipeline
│   │   ├── structural.ts
│   │   └── semantic.ts

```

```

■ ■ ■■ logical.ts
■ ■ ■■ drift.ts
■ ■■■ translator/ # IR + backend adapters
■ ■ ■■ ir.ts
■ ■ ■■ adapters/
■ ■ ■ ■■ claude.ts
■ ■ ■ ■■ cursor.ts
■ ■ ■ ■■ generic.ts
■ ■■■ registry/ # Template pack management
■ ■■■ config/ # User + project config
■ ■■■ plugin/ # Plugin API
■■■ templates/ # Built-in template packs
■■■ tests/
■ ■■■ unit/
■ ■■■ integration/
■ ■■■ e2e/
■ ■■■ fuzz/
■ ■■■ fixtures/ # 10+ representative repos
■■■ docs/
■■■ .github/workflows/

```

Configuration Hierarchy

```

# Global user config: ~/.config/bp/config.json
{
  "default_backend": "claude",
  "template_registry": "https://registry.agentic.dev",
  "custom_templates": ["~/.bp/templates/"],
  "verify_on_init": true,
  "auto_fix_level": "structural",
  "ci_mode": false
}

# Project config: .bp.json (optional, committed to repo)
{
  "backend": "claude",
  "extends": "@myorg/blueprint-base", // Inheritance
  "overrides": { "rules": { "severity_defaults": "soft" } },
  "exclude": ["legacy/", "vendor/", "dist/"],
  "plugins": ["@myorg/bp-validate-rationale"]
}

```


CI/CD Integration

```
# .github/workflows/blueprint-verify.yml
name: Blueprint Integrity
on: [push, pull_request]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: agentic-blueprint/verify@v1
    with:
      backend: claude
      level: all
      fail-on: logical # exit 4 on rule conflicts
      json-report: true # upload validation report as artifact
# .pre-commit-hooks.yaml (in bp repo)
- id: blueprint-verify
  name: Verify Agentic Blueprint
  entry: bp verify --level semantic
  language: system
  pass_filenames: false
  always_run: true
```

Distribution Channels

Channel	Package Name	Target User	Install Command
npm	@agentic/bp	JavaScript/TypeScript developers	npx @agentic/bp init
Homebrew	agentic-blueprint	macOS/Linux developers	brew install agentic-blueprint
GitHub Releases	Prebuilt binaries	CI environments, non-JS developers	curl bash installer
Docker	ghcr.io/agentic/bp	Containerized CI	docker run ghcr.io/agentic/bp verify
Cargo	cargo install bp	Rust developers (Phase 4)	cargo install bp
Winget	agentic.blueprint	Windows developers	winget install agentic.blueprint

Security Model

Threat	Mitigation
Malicious template code execution	Handlebars is logic-less; no eval or require in templates; custom helpers are allowlisted
Secrets leaked into generated files	Post-generation scan for API key patterns, JWT tokens, private key headers
Template registry supplies malicious packs	All registry packages are signed; bp verifies signature before extraction
Path traversal in template output	All output paths resolved relative to project root; validated against .. patterns
Hook files with dangerous code	bp generates stubs only — never executes hooks; execution is the agentic tool's job
Prototype pollution in template context	Template context is deep-frozen before Handlebars rendering

Performance Targets

Operation	Target	Conditions
bp init	< 2 seconds	Repos up to 1,000 files
bp verify	< 1 second	Incremental validation (only changed files)
bp verify --level all	< 5 seconds	Full validation including drift detection
bp init (10,000 file repo)	< 8 seconds	Parallel scanning enabled
Memory footprint	< 100 MB peak	During detection of large repos
Binary size	< 50 MB	Single compiled binary with all dependencies
Template render (cold)	< 500 ms	First-time rendering with cache miss
Template render (warm)	< 50 ms	Cached template compilation

SECTION 13

Testing Strategy

Test pyramid · Fixture repos · Fuzz testing · Coverage targets

Test Pyramid

Layer	Count	Type	Examples
Unit	200+	Fast, isolated	Detector heuristics, template rendering, validation rules, IR serialization
Integration	50+	Fixture repos	Full init → verify flow on all 10 real repo structures
End-to-End	20+	CLI commands	bp init --tool claude on temp dir; verify exit codes; test --dry-run diff
Fuzz	1000+	Random repos	Random file trees; ensure no panics, no hangs, no data corruption
Snapshot	100+	Template output	Rendered template files compared against golden snapshots
Round-trip	50+	Translation	claude → cursor → claude; verify semantic equivalence
Performance	10+	Large repos	10,000+ file repos; measure init and verify wall-clock time
Security	30+	Adversarial inputs	Malicious template names, path traversal attempts, oversized files

Fixture Repository Library

Fixture	Stack
node-express	Node.js + Express + TypeScript + Jest + ESLint
node-nextjs	Next.js 14 App Router + TypeScript + Vitest + Prettier
node-nestjs	NestJS + TypeScript + Jest + Swagger
node-monorepo	pnpm workspace + Turborepo + multiple packages
python-fastapi	FastAPI + Pydantic + pytest + Ruff + Poetry
python-django	Django 5 + pytest-django + Black + pip
go-std	Go stdlib + go test + golangci-lint
go-microservices	Multi-service Go monorepo + Makefile + Docker Compose
rust-axum	Axum + Tokio + cargo test + rustfmt + clippy
java-spring	Spring Boot 3 + Maven + JUnit 5
ruby-rails	Rails 7 + RSpec + Rubocop + Bundler
mixed-language	Python backend + TypeScript frontend + Docker Compose

Quality Gates (CI Enforcement)

- Coverage $\geq$ 95% (Codecov): PR fails if coverage drops below threshold
- Zero known false positives: fixture test suite must pass with zero unexpected errors

- Detection accuracy $\geq 95\%$: validated against 50 real open-source repositories
- Template render correctness 100%: all snapshot tests pass on every PR
- Round-trip fidelity $\geq 98\%$: translation round-trip tests pass
- No performance regressions: benchmark suite runs on every merge to main
- Security scan: Snyk + CodeQL on every PR; zero high-severity findings to merge

Completeness Audit: Gaps in the Original Plan

The following gaps were identified in the v1.0 plan and have been addressed in this expanded document:

Gap	Resolution in This Plan
Missing Languages	Original plan listed 10 languages; this plan adds Swift, C#, Kotlin, and Dart with full detection signals
Security Signals in Fingerprint	Original fingerprint schema had no security context; added <code>has_auth</code> , <code>has_external_apis</code> , <code>has_secrets_manager</code> , <code>has_docker</code>
No Hook Authoring Guide	Hooks were mentioned but never specified; full stub generation and safety validation added
Plugin API Absent	Original plan had no extensibility mechanism; full TypeScript plugin API with custom validators added
IDE Integration Unspecified	Original mentioned VS Code extension as open question; now fully scoped in Phase 4
Missing Windows Support	Distribution was macOS/Linux focused; Winget and Windows binaries added
No LSP Specification	IDE integration was vague; blueprint-lsp server specified for editor-agnostic support
Rule graph visualization missing	bp rule graph command added for ASCII coverage map
Hook validation unspecified	bp hook validate command added to check stub safety before commit
Config management commands missing	bp config get/set/reset added for managing user and project config
Dependency drift not covered	Drift engine extended to detect new major dependencies with no skill coverage
No Goose backend	Goose (by Block) added as Phase 4 translation target
Registry auth not specified	Template publish flow and auth token handling fully specified
No audit logging	Enterprise audit log of all bp operations added in Phase 4
Semantic NLP contradiction detection	Original listed as open question; embedding-based detection scheduled for v2

v2 Roadmap (Post-Production)

- **AI-Assisted Template Generation:** Use an LLM to analyze a repo's existing conventions (README, CONTRIBUTING, eslint config) and generate a custom template pack without any manual authoring.

- **Semantic Rule NLP / Embeddings:** Use vector embeddings to detect semantic contradictions between rules beyond keyword antonym matching — e.g. 'prefer functional style' vs 'use OOP patterns'.
- **Dynamic Rule Generation from Code Review:** Analyze merged PR comments and code review patterns to suggest new rules that encode team conventions not yet captured in the blueprint.
- **Cross-Repository Governance Dashboard:** Web UI showing blueprint health across all repos in an organization — coverage scores, drift alerts, conflict counts, adoption metrics.
- **Blueprint Diff / Migration Tool:** When an org updates its base template, bp migrate shows a diff and applies changes repo-by-repo with interactive approval.
- **Real-time Agentic Loop Integration:** Optional lightweight sidecar that watches the agentic tool's output and flags live violations of hard rules — without blocking the loop.

Success Metrics

Metric	6-Month Target	12-Month Target	Measurement
npm weekly downloads	1,000	10,000	Codecov badge on README
GitHub stars	500	2,500	Star history chart
Active template packs (community)	10	50	Registry explorer page
Organization adopters	5	50	Case studies / logo wall
CI integrations	100 repos	1,000 repos	GitHub Action marketplace installs
Blueprints with zero validation errors	> 80%	> 95%	Telemetry (opt-in)
False positive rate	< 5/month	< 1/month	GitHub issues labeled false-positive